

III užduotis

**Vaizdų klasifikavimas naudojant
konvoliucinius neuroninius tinklus**

Atsiskaitymas

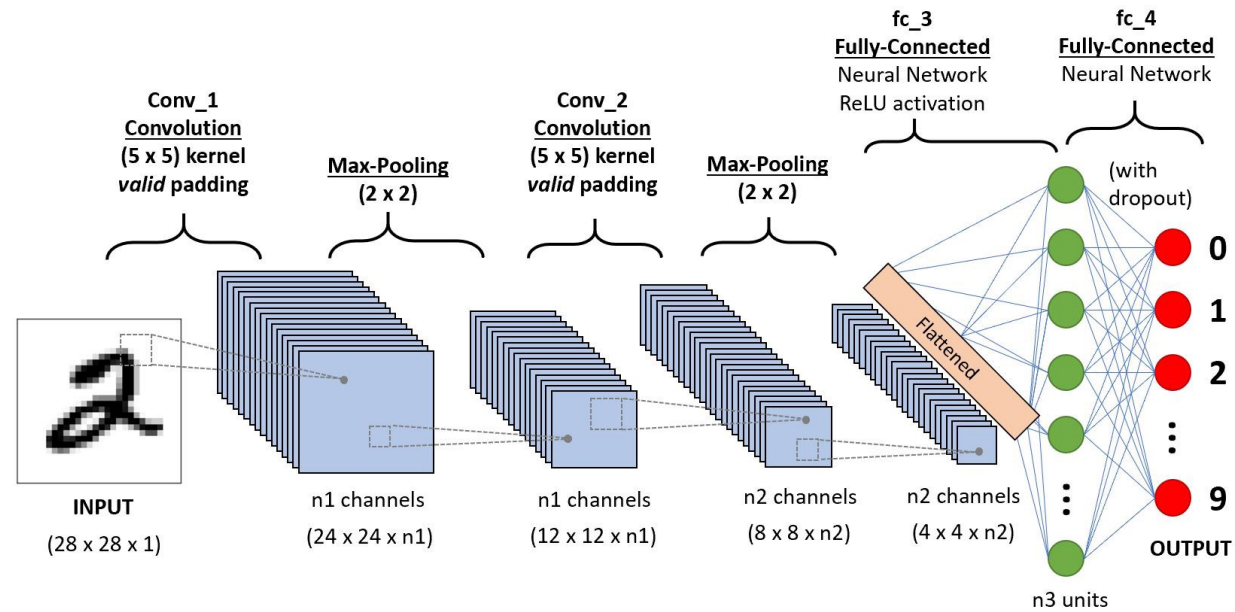
Galutinio įvertinimo sudedamosios dalys:

Praktinės užduotys (semestro eigoje) – 5 balai,

Kolokviumas (semestro viduryje) – 2 balai,

Egzaminas (sesijos metu) – 3 balai.

Studentas įgyja teisę laikyti egzaminą tik tuo atveju, jeigu jis už praktines užduotis surenka nemažiau kaip 50 % galimų balų.



<https://towardsdatascience.com/reducing-the-carbon-foot-prints-of-cnns-at-the-cost-of-interactions-depthwise-pointwise-conv-5df850ea33a4>

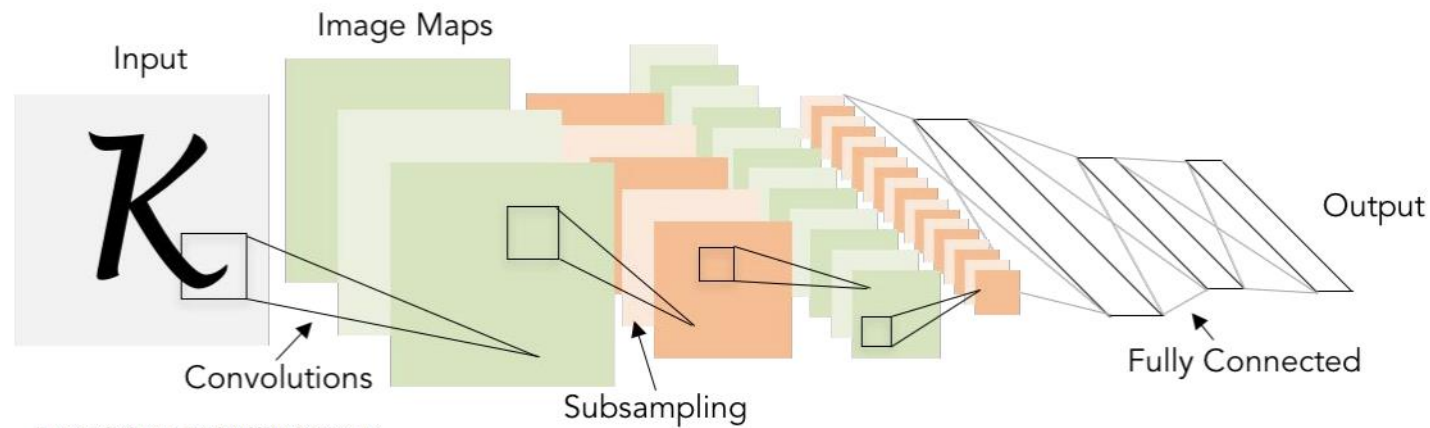


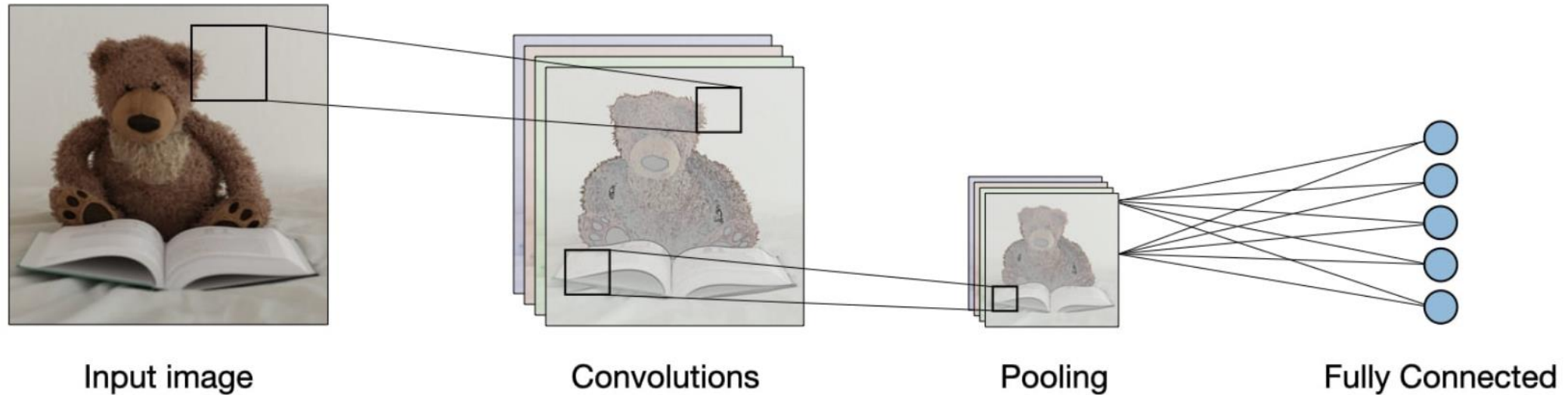
Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

http://cs231n.stanford.edu/slides/2023/lecture_5.pdf

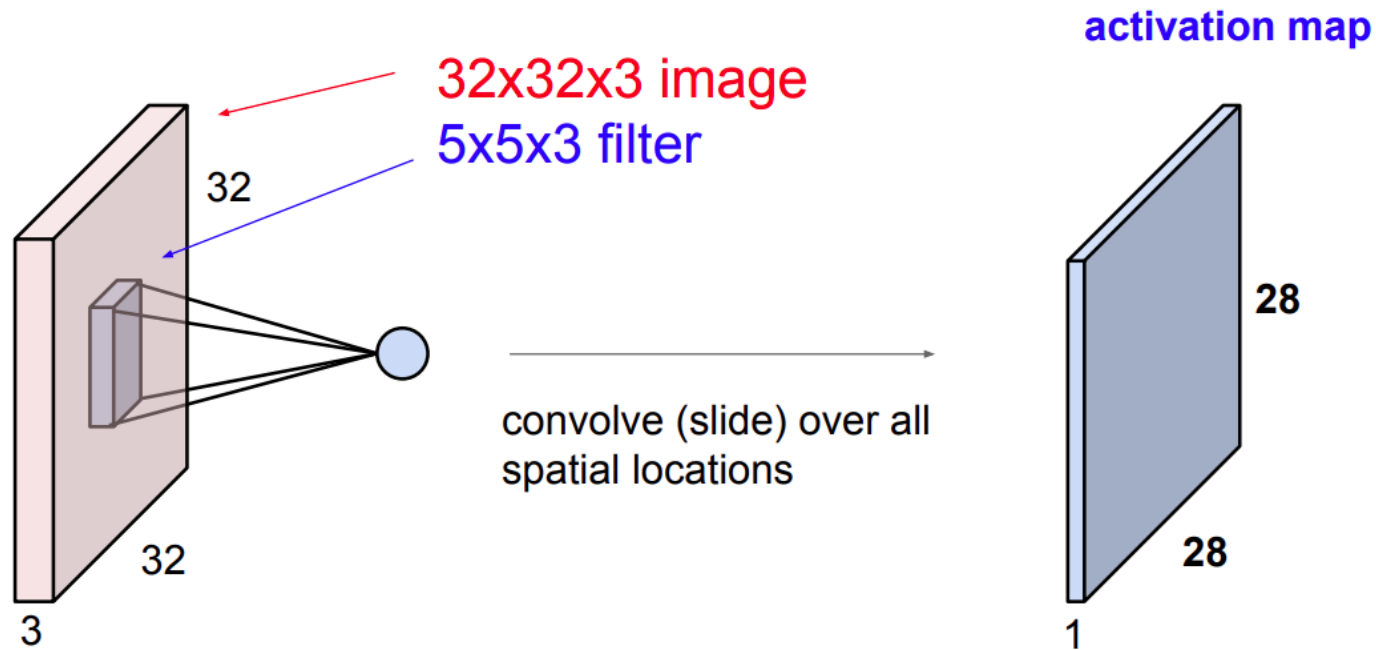


<https://bbycroft.net/llm>

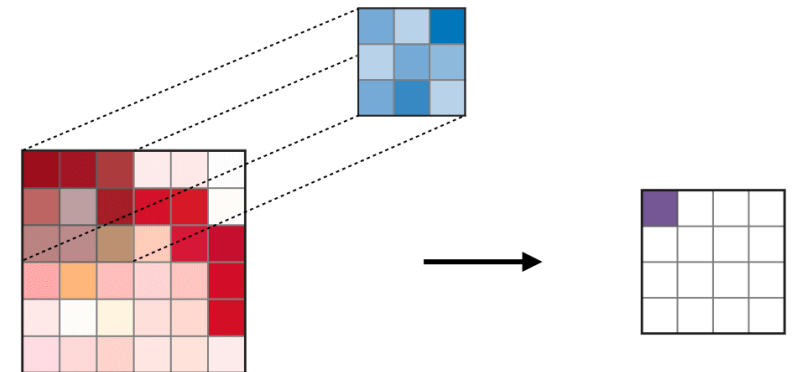
❑ **Architecture of a traditional CNN** — Convolutional neural networks, also known as CNNs, are a specific type of neural networks that are generally composed of the following layers:



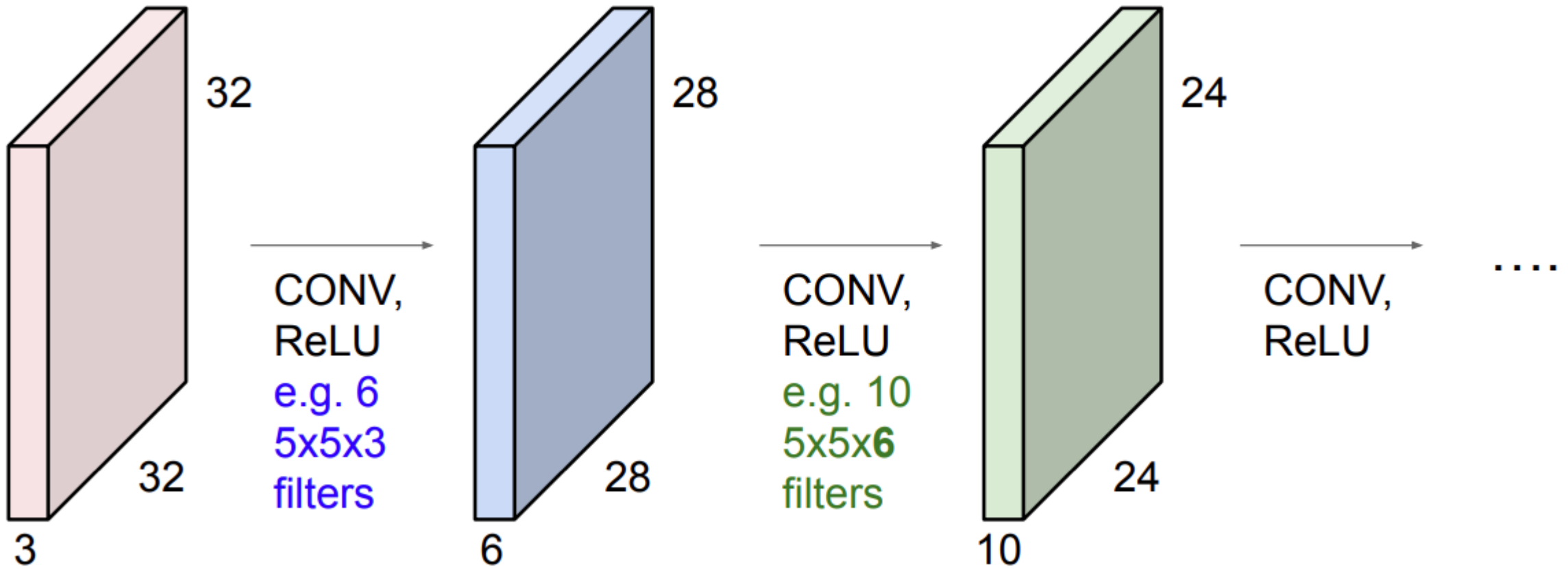
A closer look at spatial dimensions:



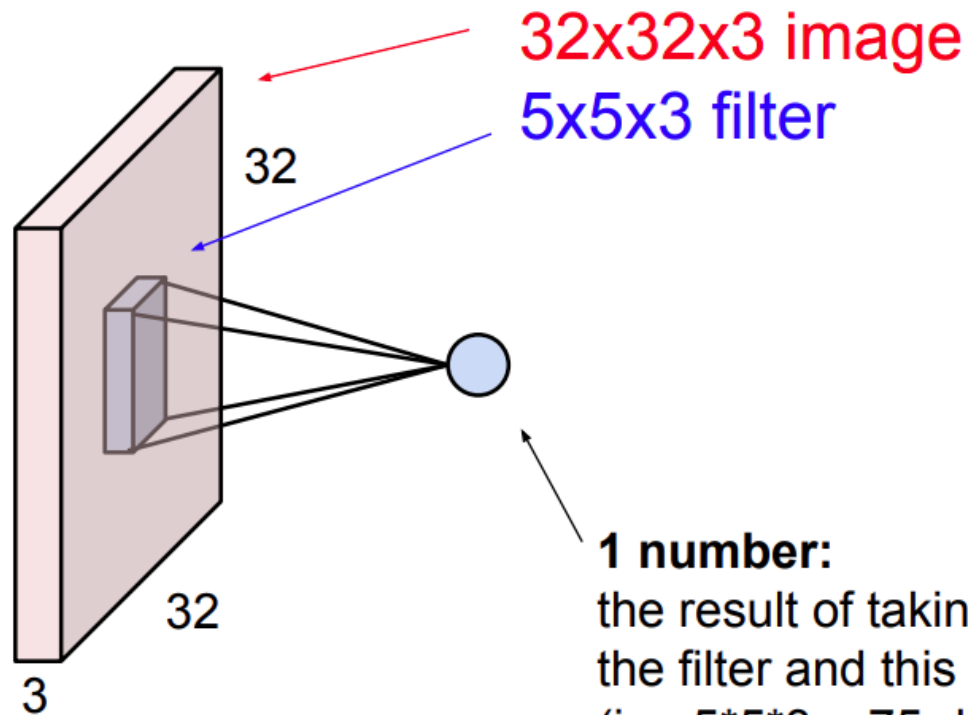
http://cs231n.stanford.edu/slides/2023/lecture_5.pdf



<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>

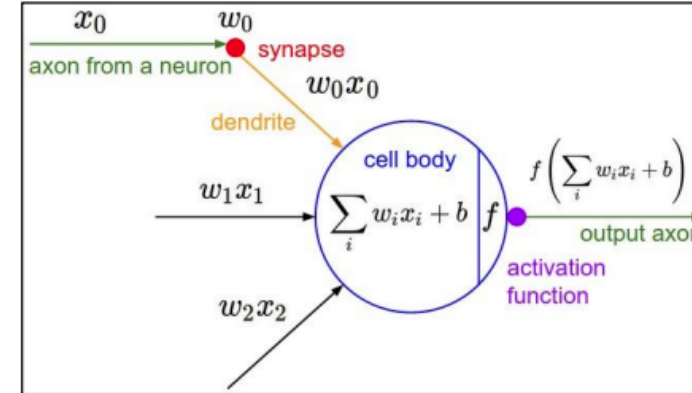


The brain/neuron view of CONV Layer

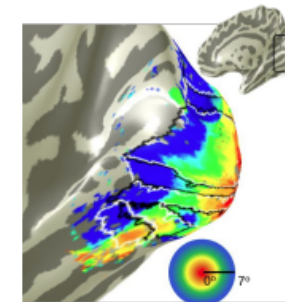


1 number:

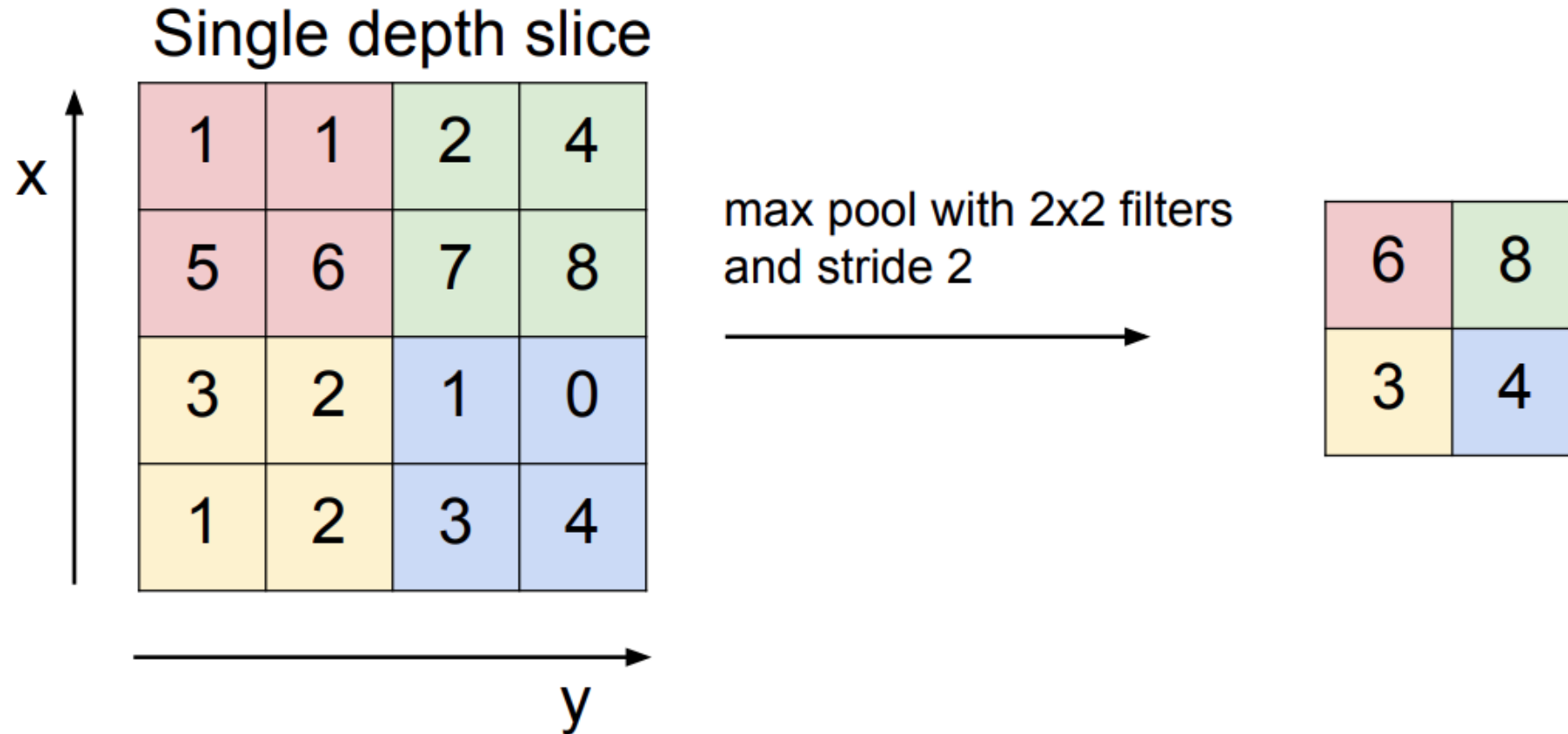
the result of taking a dot product between the filter and this part of the image (i.e. $5 \times 5 \times 3 = 75$ -dimensional dot product)



It's just a neuron with local connectivity...

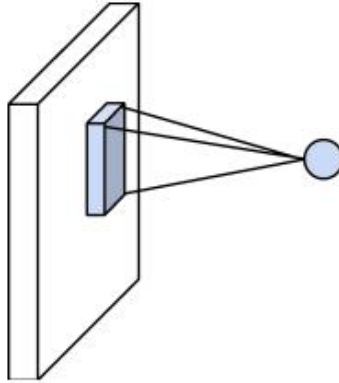


MAX POOLING

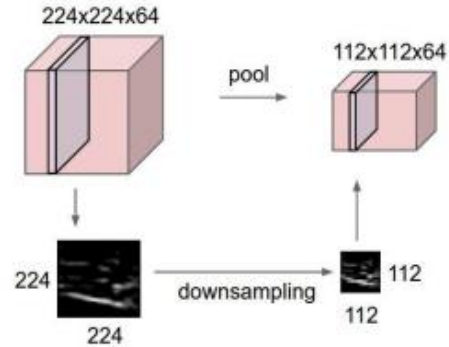


Components of CNNs

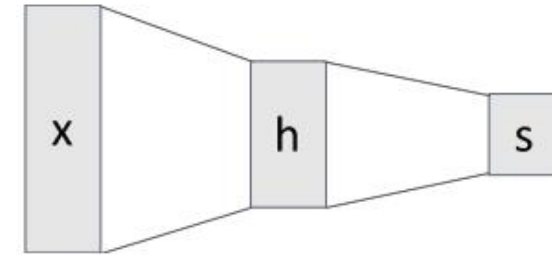
Convolution Layers



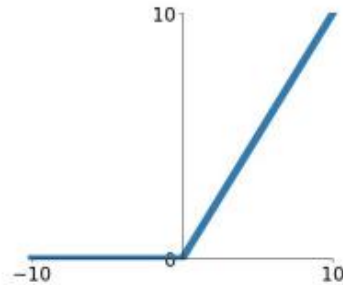
Pooling Layers



Fully-Connected Layers

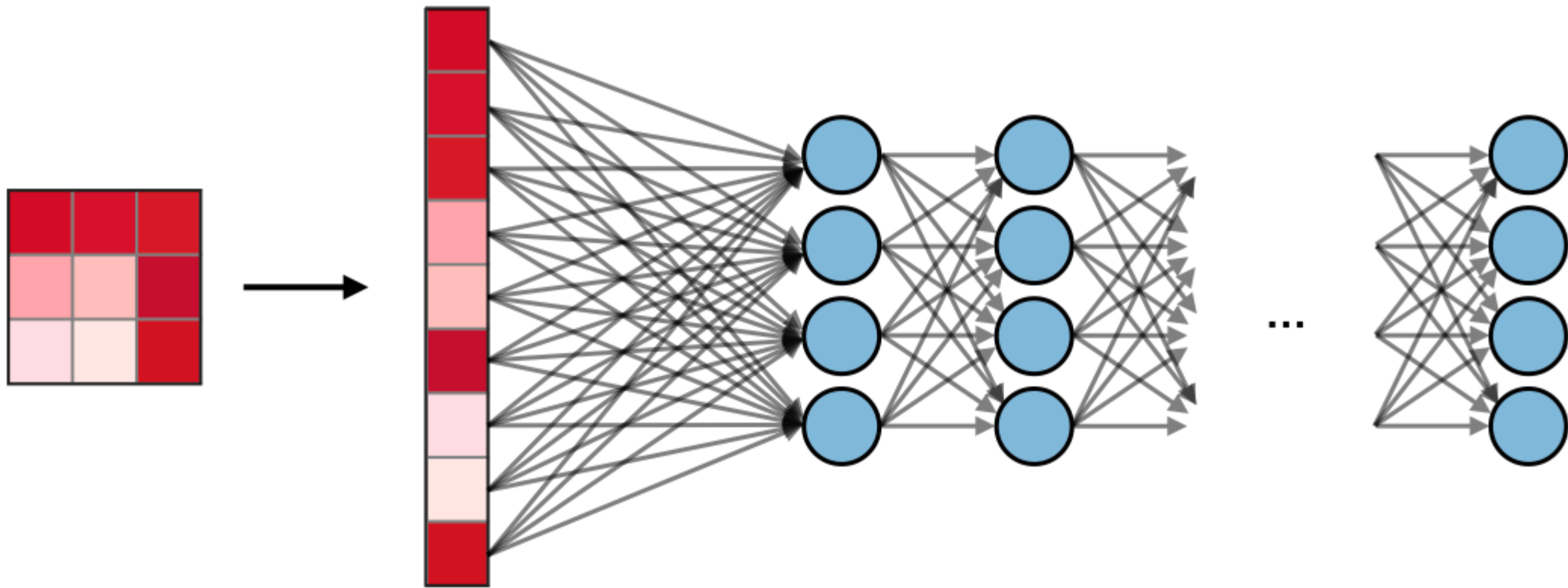


Activation Function



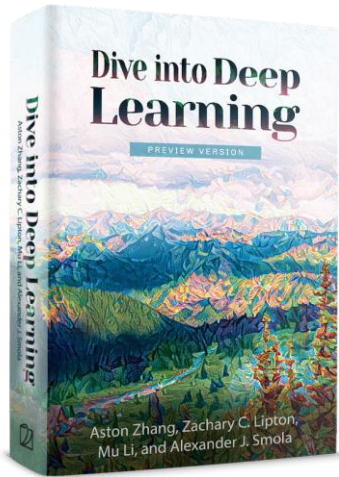
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$



<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>

Nuorodos



Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2021). Dive into deep learning.*arXiv preprint arXiv:2106.11342*.
<http://d2l.ai/index.html>

Nuorodos:

Stanford CS class CS231n: Convolutional Neural Networks for Visual Recognition:

<https://www.youtube.com/playlist?list=PLkt2uSq6rBVctENoVBg1TpCC7OQi31AIC> 

<https://cs231n.github.io/>



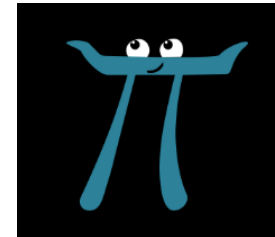
CS231n Convolutional Neural Networks for Visual Recognition

(2023) <http://cs231n.stanford.edu/schedule.html>

<https://cs231n.github.io/neural-networks-2/>

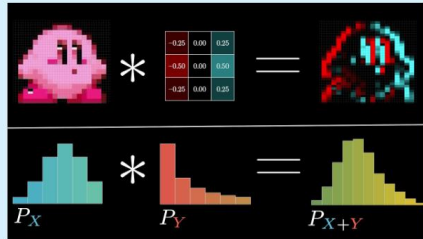
<https://cs231n.github.io/neural-networks-3/>

https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi 



3blue1brown: <https://www.3blue1brown.com/lessons/neural-networks>

Nuorodos:



But what is a convolution?

From probability to image processing and FFTs, an overview of discrete convolutions

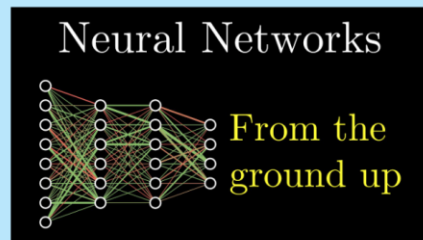
17 Nov 2022



But what is a GPT? Visual intro to Transformers | Deep learning, chapter 5

A visual introduction to transformers. This chapter focusses on the overall structure, and word...

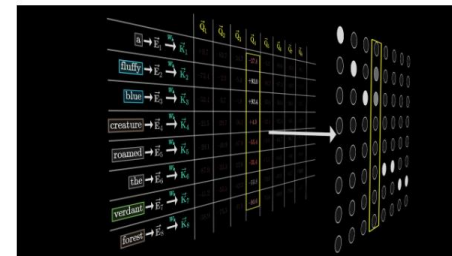
1 Apr 2024



But what is a Neural Network?

An overview of what a neural network is, introduced in the context of recognizing hand-written digits.

Chapter 1 5 Oct 2017



Visualizing Attention, a Transformer's Heart | Chapter 6, Deep Learning

Demystifying attention, the key mechanism inside transformers and LLMs.

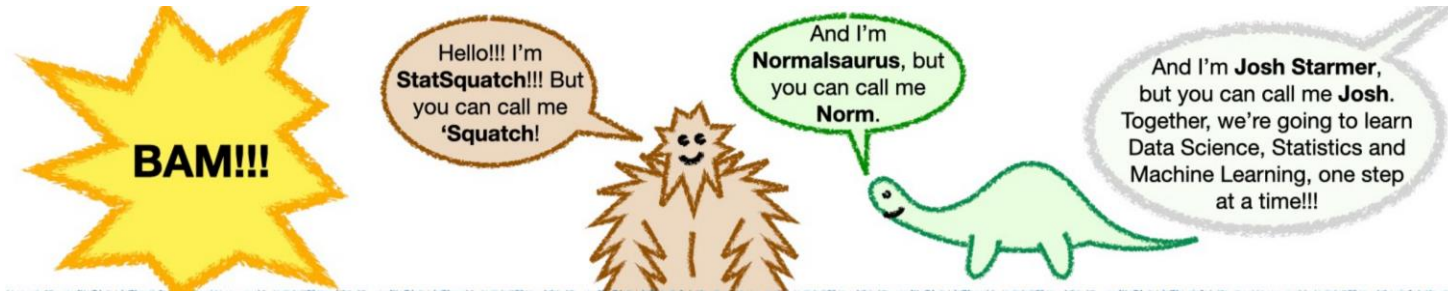
7 Apr 2024

https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi



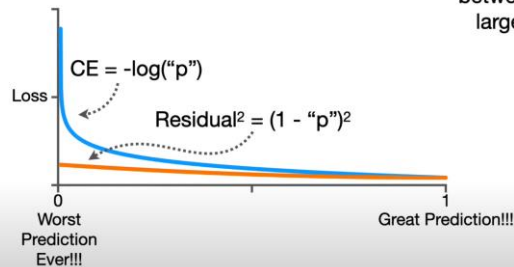
3blue1brown: <https://www.3blue1brown.com/lessons/neural-networks>

Nuorodos:



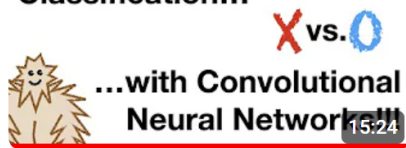
<https://www.youtube.com/@statquest>

Petal	Sepal	Species	"p"	Cross Entropy
0.04	0.42	Setosa	0.57	$-\log("p") = 0.56$



...the change in **Loss** between **0** and **1** is not as large as it is for **Cross Entropy**.

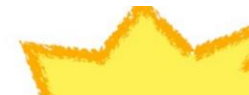
Deep Learning Image Classification...



Neural Networks Part 8: Image Classification with Convolutional Neural Networks (CNNs)

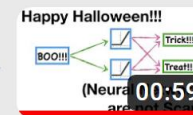
StatQuest with Josh Starmer • 205 tūkst. peržiūrų • prieš 3 metus

One of the coolest things that Neural Networks can do is classify images, and this is often done with a type of Neural Network called a Convolutional Neural Network (or CNN for short). In this...



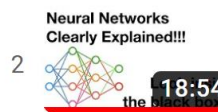
Neural Networks / Deep Learning

StatQuest with Josh Starmer - 1 / 24



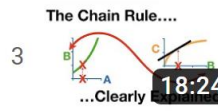
Happy Halloween (Neural Networks Are Not Scary)

StatQuest with Josh Starmer



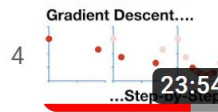
Neural Networks Pt. 1: Inside the Black Box

StatQuest with Josh Starmer



The Chain Rule

StatQuest with Josh Starmer



Gradient Descent, Step-by-Step

StatQuest with Josh Starmer



Neural Networks Pt. 2: Backpropagation Main Ideas

StatQuest with Josh Starmer



Backpropagation Details Pt. 1: Optimizing 3 parameters...

StatQuest with Josh Starmer

HOW TO CONFUSE MACHINE LEARNING



<https://www.kaggle.com/datasets/samuelcortinhas/muffin-vs-chihuahua-image-classification/data>

IV užduotis (Vaizdų ir laiko eilučių klasifikavimas naudojant konvoliucinius neuroninius tinklus)

Užduotyje bus naudojami dviejų tipų duomenys: vaizdai ir laiko eilutės.

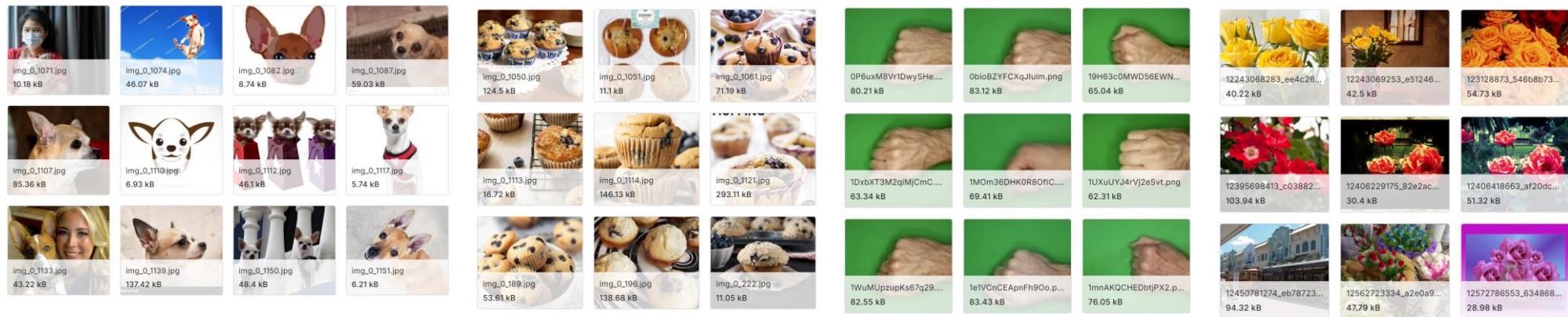
Tikslas – apmokyti konvoliucinius neuroninius tinklus vaizdams ir laiko eilutėms klasifikuoti, atlikti tyrimą. Vaizdams reikia taikyti 2D konvoliucijas, laiko eilutėms – 1D konvoliucijas.

Vaizdų duomenys:

Tyrimui pasirinkite vieną iš nurodytų duomenų rinkinių (vaizdus su žinomomis klasėmis):

- Keksiukų ir šuniukų vaizdai:
<https://www.kaggle.com/datasets/samuelcortinhas/muffin-vs-chihuahua-image-classification/data>
- Žaidimo „Akmuo-popierius-žirkklės“ vaizdai:
<https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors/data>
- Gėlių vaizdai:
<https://www.kaggle.com/datasets/sauravagarwal/flower-classification>

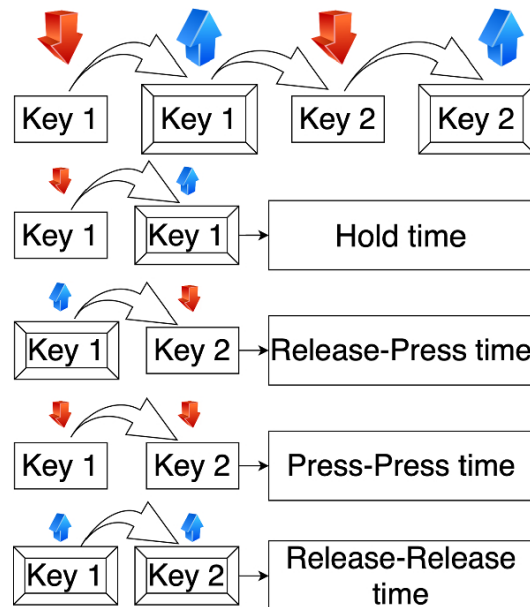
Jei turite ribotus skaičiavimo resursus, galite imti ne visą vaizdų aibę, o tik jos dalį. Taip pat galima sumažinti klasių skaičių, t. y., analizuoti ne visas klases.



Laiko eilučių duomenys:

Tyrimui naudokite CMU duomenų aibę, kurią galima atsisiųsti iš <https://www.cs.cmu.edu/~keystroke/>. Tai klavišų paspaudimų, renkant fiksuotą slaptažodį „tie5Roanl“, duomenų aibė. Buvo įrašomas klavišų paspaudimų laikas, klavišų atleidimo laikas bei laikas tarp klavišų paspaudimų. Gautos laiko eilutės, kurių požymių skaičius yra 31. Eksperimente dalyvavo 57 dalyviai, vyko 8 sesijos, kiekvienos sesijos metu dalyvis turėjo surinkti slaptažodį 50 kartų. Ši duomenų aibė yra naudojama norint nustatyti, kaip skiriasi skirtingų žmonių klavišų paspaudimų bruožai. Tai yra svarbu, norint identifikuoti insaiderius (neteisėtus naudotojus) pagal biometrinius duomenis.

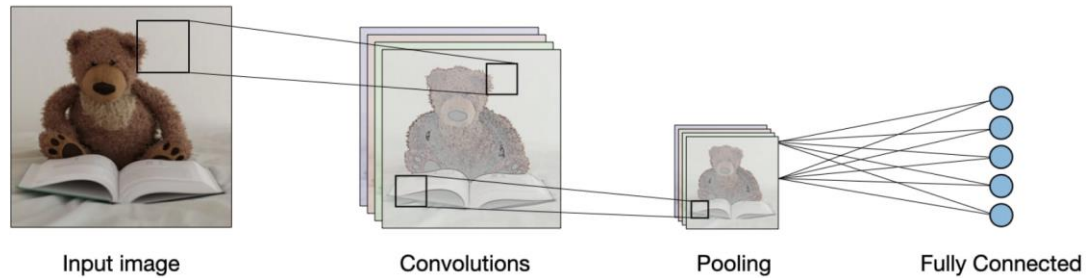
Užduočiai atlikti reikia pasirinkti 30 bet kurių naudotojų, pasirinkti vieną bet kurią sesiją. Užduoties tikslas – suklasifikuoti duomenis į 30 klasių.

[illegible]

<https://link.springer.com/article/10.1007/s10462-024-10893-1>

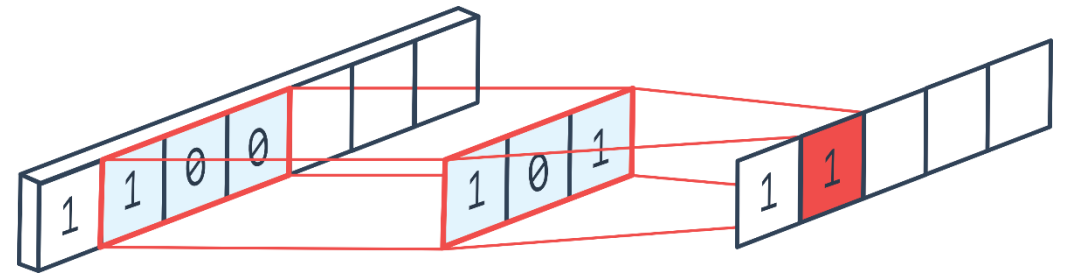
Conv 2d vs Conv1d

❑ **Architecture of a traditional CNN** — Convolutional neural networks, also known as CNNs, are a specific type of neural networks that are generally composed of the following layers:



The convolution layer and the pooling layer can be fine-tuned with respect to hyperparameters that are described in the next sections.

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>



<https://stackoverflow.com/questions/6500601/size-of-output-of-a-conv1d-layer-in-keras>

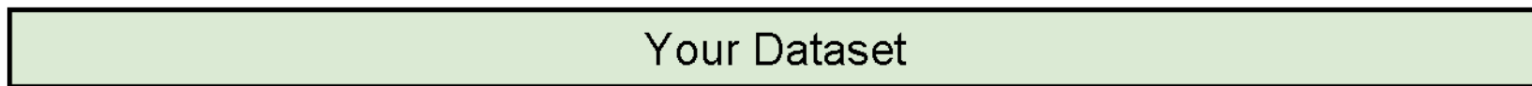
Užduoties punktai:

1. Paruoškite duomenis konvoliuciniam neuroniniam tinklui apmokyti. Jei duomenys jau yra suskirstyti į mokymo, validavimo ir testavimo, turite juos apjungti į vieną rinkinį ir pagal norimą santykį (pvz., 80:10:10) juos perskirstyti į mokymo, validavimo ir testavimo.

Setting Hyperparameters

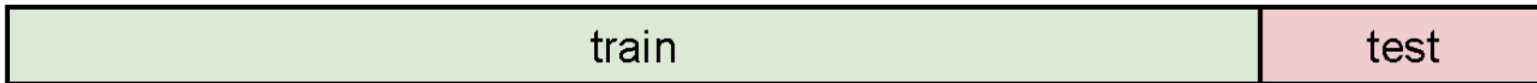
Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

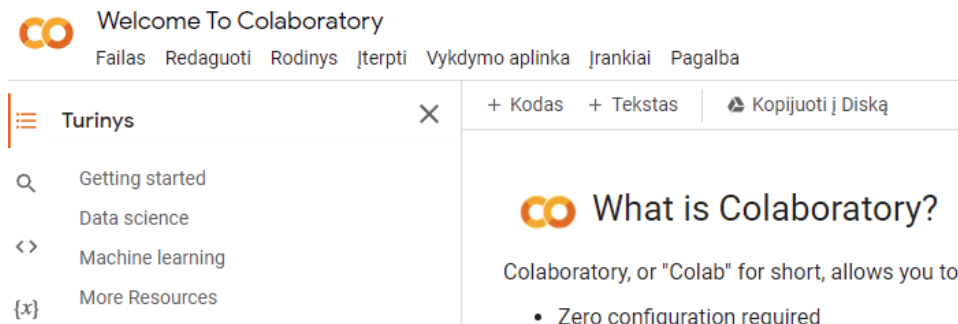
Better!



2. Sukurti programą (programinį kodą), iš kurių vienoje įgyvendintas konvoliucinis neuroninis tinklas vaizdams klasifikuoti (taikyti 2D konvoliucijas), kitame – laiko eilutėms klasifikuoti (taikyti 1D konvoliucijas). Galima naudoti *TensorFlow*, *Keras*, *PyTorch*, *Jupyter Notebook* ar kitus reikiamus įrankius. Galima naudotis *Google Colab* platforma, kurioje galima rašyti bei vykdyti *Python* kodą per naršyklę. Programose turi būti galimybė:

- Keisti tinklo architektūrą (kode pridedant/panaikinant įvairius sluoksnius),
- Keisti hiperparametrų reikšmes (aktyvacijos funkciją, epochų skaičių, mokymosi greitį, paketo dydį (*batch size*), branduolio (*kernel*) dydį, branduolių kiekį konvoliuciniuose sluoksniuose, filtro dydį sujungimo (*pooling*) sluoksniuose, išmetimo sluoksnio (*dropout*) tikimybę, optimizavimo algoritmą, nuostolių (*loss*) funkciją ir kt.),
- Apskaičiuoti gautą paklaidą (*loss*),
- Apskaičiuoti klasifikavimo tikslumą (*accuracy*),
- Apskaičiuoti klasifikavimo matricą (*confusion matrix*).

Galima naudoti **Google Colaboratory**:
colab.research.google.com/



model.summary()

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1179776
dense_1 (Dense)	(None, 10)	1290

=====

Total params: 1,199,882
Trainable params: 1,199,882
Non-trainable params: 0



TensorFlow



Get started with **Google Colaboratory (Coding TensorFlow)**: <https://www.youtube.com/watch?v=inN8seMm7UI>



<https://colab.research.google.com/notebooks/intro.ipynb>

https://colab.research.google.com/notebooks/basic_features_overview.ipynb



Welcome to Colaboratory

File Edit View Insert Runtime Tools Help

<https://cs231n.github.io/setup-instructions/>

CS231n Convolutional Neural Networks for Visual Recognition

Python Numpy Tutorial (with Jupyter and Colab): <https://cs231n.github.io/python-numpy-tutorial/>

Galima naudoti **Google Colaboratory**:
colab.research.google.com/



Welcome To Colaboratory

Failas Redaguoti Rodinys Įterpti Vykdymo aplinka Įrankiai Pagalba



Turinys



+ Kodas

+ Tekstas

Kopijuoti į Diską



Getting started



Data science



Machine learning



More Resources



What is Colaboratory?

Colaboratory, or "Colab" for short, allows you to

- Zero configuration required



model.summary()



Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1179776
dense_1 (Dense)	(None, 10)	1290

=====
Total params: 1,199,882
Trainable params: 1,199,882
Non-trainable params: 0



TensorFlow



Keras

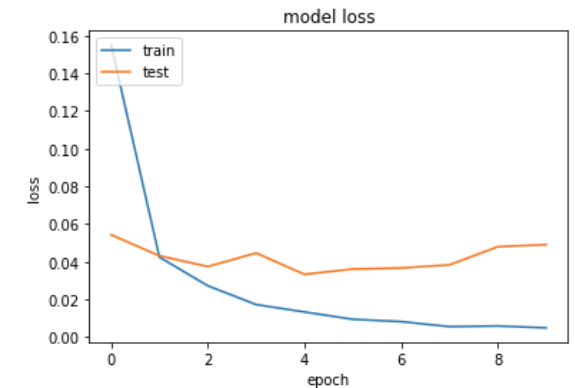
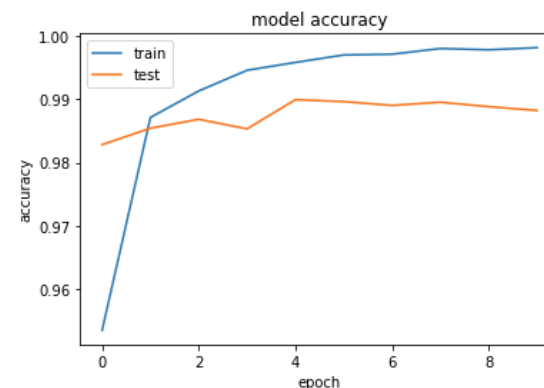


PyTorch

2. Sukurti programas (programinį kodą), iš kurių vienoje įgyvendintas konvoliucinis neuroninis tinklas vaizdams klasifikuoti (taikyti 2D konvoliucijas), kitame – laiko eilutėms klasifikuoti (taikyti 1D konvoliucijas). Galima naudoti *TensorFlow*, *Keras*, *PyTorch*, *Jupyter Notebook* ar kitus reikiamus įrankius. Galima naudotis *Google Colab* platforma, kurioje galima rašyti bei vykdyti *Python* kodą per naršyklę. Programose turi būti galimybė:

- Keisti tinklo architektūrą (kode pridedant/panaikinant įvairius sluoksnius),
- Keisti hiperparametrų reikšmes (aktyvacijos funkciją, epochų skaičių, mokymosi greitį, paketo dydį (*batch size*), branduolio (*kernel*) dydį, branduolių kiekį konvoliuciniuose sluoksniuose, filtro dydį sujungimo (*pooling*) sluoksniuose, išmetimo sluoksnio (*dropout*) tikimybę, optimizavimo algoritmą, nuostolių (*loss*) funkciją ir kt.),
- Apskaičiuoti gautą paklaidą (*loss*),
- Apskaičiuoti klasifikavimo tikslumą (*accuracy*),
- Apskaičiuoti klasifikavimo matricą (*confusion matrix*).

```
Epoch 1/10
469/469 [=====] - 38s 17ms/step - loss: 0.1551 - accuracy: 0.9535 - val_loss: 0.0543 - val_accuracy: 0.9828
Epoch 2/10
469/469 [=====] - 7s 16ms/step - loss: 0.0424 - accuracy: 0.9871 - val_loss: 0.0432 - val_accuracy: 0.9854
Epoch 3/10
469/469 [=====] - 7s 16ms/step - loss: 0.0273 - accuracy: 0.9913 - val_loss: 0.0375 - val_accuracy: 0.9868
Epoch 4/10
469/469 [=====] - 7s 15ms/step - loss: 0.0173 - accuracy: 0.9945 - val_loss: 0.0446 - val_accuracy: 0.9853
Epoch 5/10
.....
```



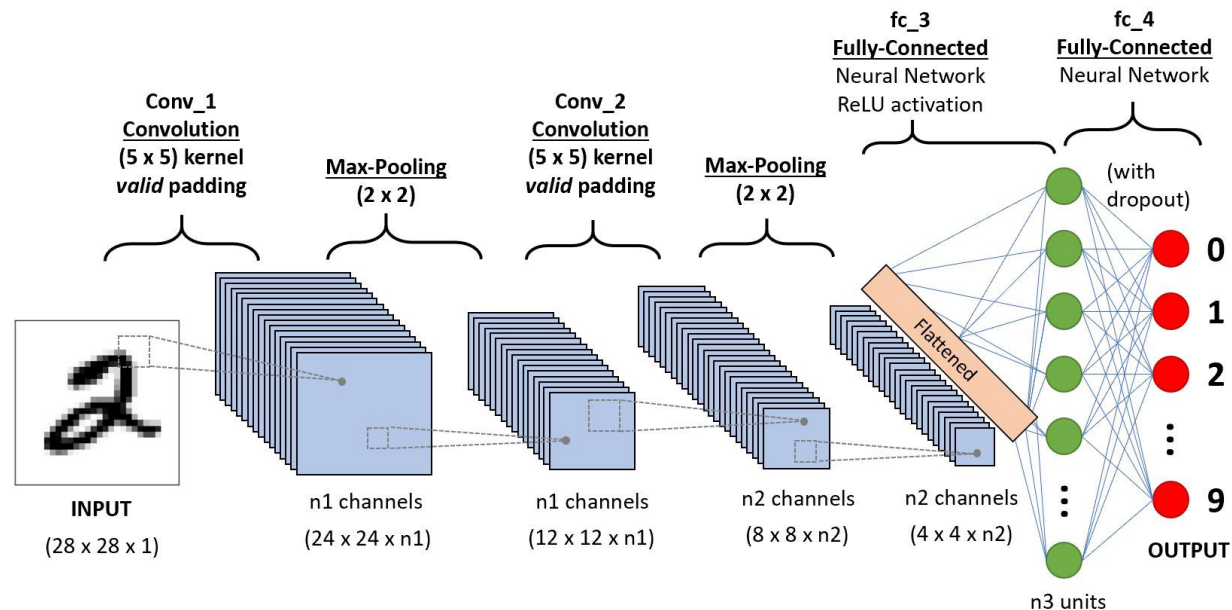
3. Kiekvienai duomenų aibei atlikti tyrimą, rezultatus pateikiant grafikuose (rezultatus vertinti pagal mokymo ir validavimo duomenų rinkinius):

- Kaip rezultatai (klasifikavimo tikslumas ir paklaida) priklauso nuo tinklo architektūros? Būtina atlikti tyrimus su mažiausiai trimis architektūromis, siekiant rasti tą, kuri leidžia gauti tiksliausius rezultatus.
- Kaip rezultatai (klasifikavimo tikslumas ir paklaida) priklauso nuo hiperparametrų reikšmių:
 - kaip keičiasi klasifikavimo tikslumas ir paklaida, jeigu pridėsime vieną arba kelis išmetimo (*dropout*) sluoksnius su įvairiomis išmetimo tikimybėmis,
 - kaip klasifikavimo tikslumą ir paklaidą įtakoja paketų normalizavimas (*batch normalization*),
 - kaip klasifikavimo tikslumą ir paklaidą įtakoja naudojama aktyvacijos funkcija,
 - kaip klasifikavimo tikslumą ir paklaidą įtakoja optimizavimo algoritmas.

P. S. Grafikuose pateikti gautų rezultatų tikslumo priklausomybės nuo analizuojamų hiperparametrų (mokymosi greičio, aktyvacijos funkcijos, epochų/iteracijų skaičiaus ir kt.) mokymo ir validavimo duomenų rinkiniams.

4. Nustačius atvejį, kada gaunamas didžiausias klasifikavimo tikslumas ir mažiausia paklaida mokymo duomenims, įvertinti testavimo duomenų klasifikavimo tikslumą ir apskaičiuoti klasifikavimo matricą (*confusion matrix*).

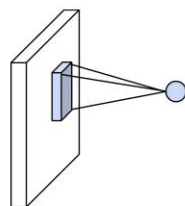
```
[7 2 1 ... 4 5 6]
[7 2 1 ... 4 5 6]
[[ 977 0 0 0 0 0 1 1 1 0]
 [ 0 1130 0 0 1 0 2 1 0 1]
 [ 2 4 1010 1 2 0 1 9 3 0]
 [ 0 0 0 1005 0 2 0 2 1 0]
 [ 0 0 0 0 970 0 4 0 1 7]
 [ 1 0 1 10 0 874 6 0 0 0]
 [ 4 2 0 0 1 1 950 0 0 0]
 [ 0 1 2 0 0 0 0 1024 1 0]
 [ 2 1 1 1 0 2 2 2 960 3]
 [ 0 2 0 4 8 1 0 11 1 982]]
```



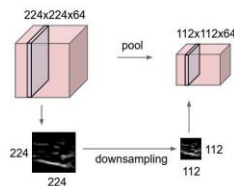
<https://towardsdatascience.com/reducing-the-carbon-foot-prints-of-cnns-at-the-cost-of-interactions-depthwise-pointwise-conv-5df850ea33a4>

Components of CNNs

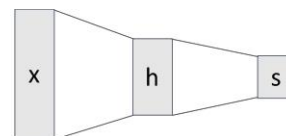
Convolution Layers



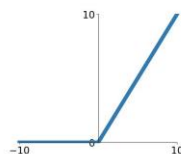
Pooling Layers



Fully-Connected Layers



Activation Function



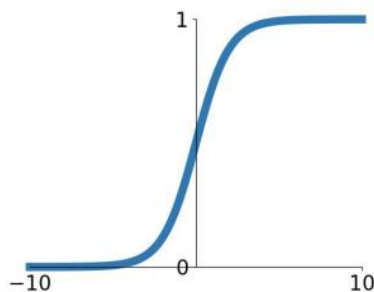
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

http://cs231n.stanford.edu/slides/2023/lecture_5.pdf

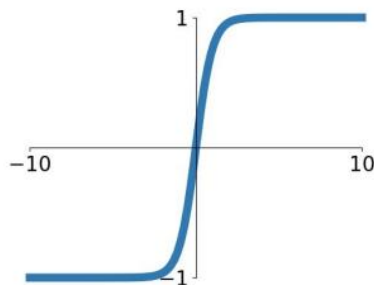
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

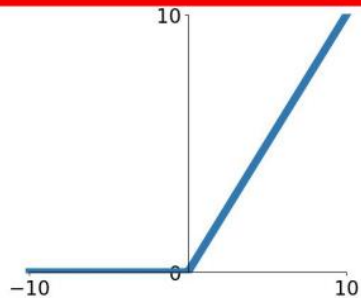
$$\tanh(x)$$



ReLU

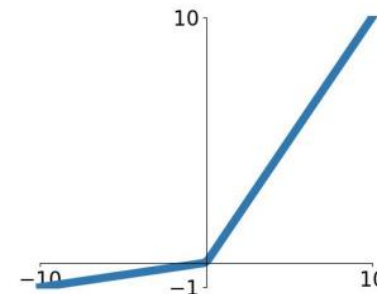
$$\max(0, x)$$

Good default choice



Leaky ReLU

$$\max(0.1x, x)$$

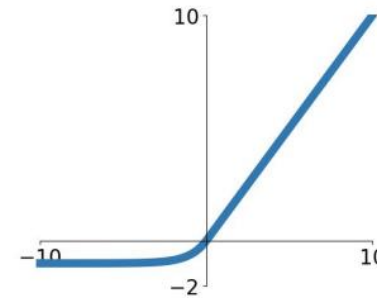


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

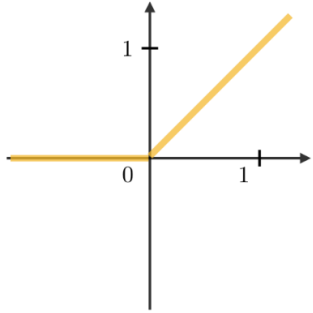
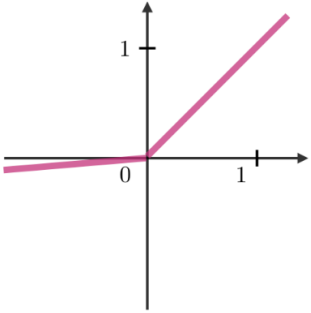
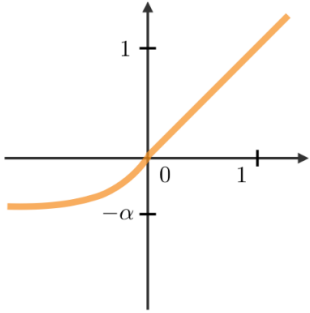
ELU

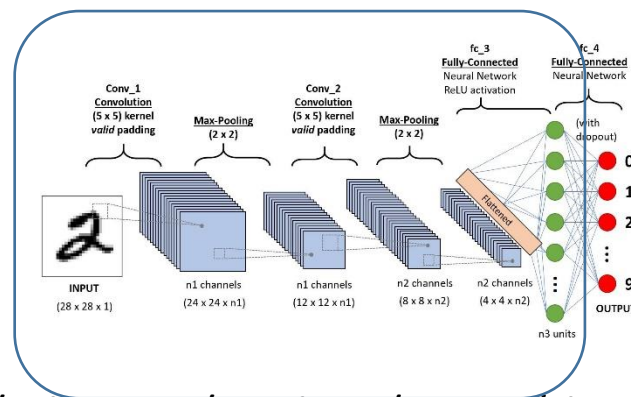
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Commonly used activation functions

□ **Rectified Linear Unit** — The rectified linear unit layer (ReLU) is an activation function g that is used on all elements of the volume. It aims at introducing non-linearities to the network. Its variants are summarized in the table below:

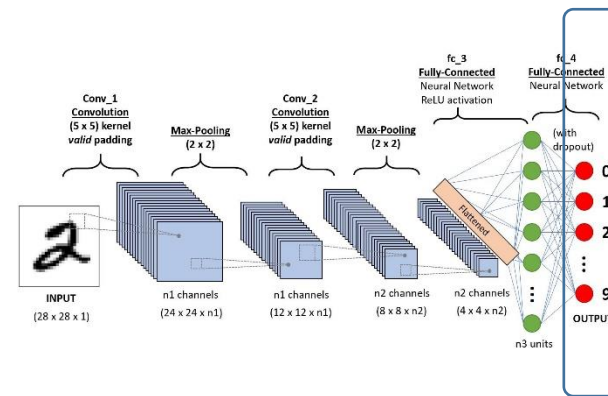
ReLU	Leaky ReLU	ELU
$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$	$g(z) = \max(\alpha(e^z - 1), z)$ with $\alpha \ll 1$
		
• Non-linearity complexities biologically interpretable	• Addresses dying ReLU issue for negative values	• Differentiable everywhere



<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>

□ **Softmax** — The softmax step can be seen as a generalized logistic function that takes as input a vector of scores $x \in \mathbb{R}^n$ and outputs a vector of output probability $p \in \mathbb{R}^n$ through a softmax function at the end of the architecture. It is defined as follows:

$$p = \begin{pmatrix} p_1 \\ \vdots \\ p_n \end{pmatrix} \quad \text{where} \quad p_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>

Optimizing convergence

❑ **Learning rate** — The learning rate, often noted α or sometimes η , indicates at which pace the weights get updated. It can be fixed or adaptively changed. The current most popular method is called Adam, which is a method that adapts the learning rate.

❑ **Adaptive learning rates** — Letting the learning rate vary when training a model can reduce the training time and improve the numerical optimal solution. While Adam optimizer is the most commonly used technique, others can also be useful. They are summed up in the table below:

Method	Explanation	Update of w	Update of b
Momentum	• Dampens oscillations • Improvement to SGD • 2 parameters to tune	$w - \alpha v_{dw}$	$b - \alpha v_{db}$
RMSprop	• Root Mean Square propagation • Speeds up learning algorithm by controlling oscillations	$w - \alpha \frac{dw}{\sqrt{s_{dw}}}$	$b \leftarrow b - \alpha \frac{db}{\sqrt{s_{db}}}$
Adam	• Adaptive Moment estimation • Most popular method • 4 parameters to tune	$w - \alpha \frac{v_{dw}}{\sqrt{s_{dw}} + \epsilon}$	$b \leftarrow b - \alpha \frac{v_{db}}{\sqrt{s_{db}} + \epsilon}$

Remark: other methods include Adadelata, Adagrad and SGD.

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks>

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See [section 2](#) for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks>

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

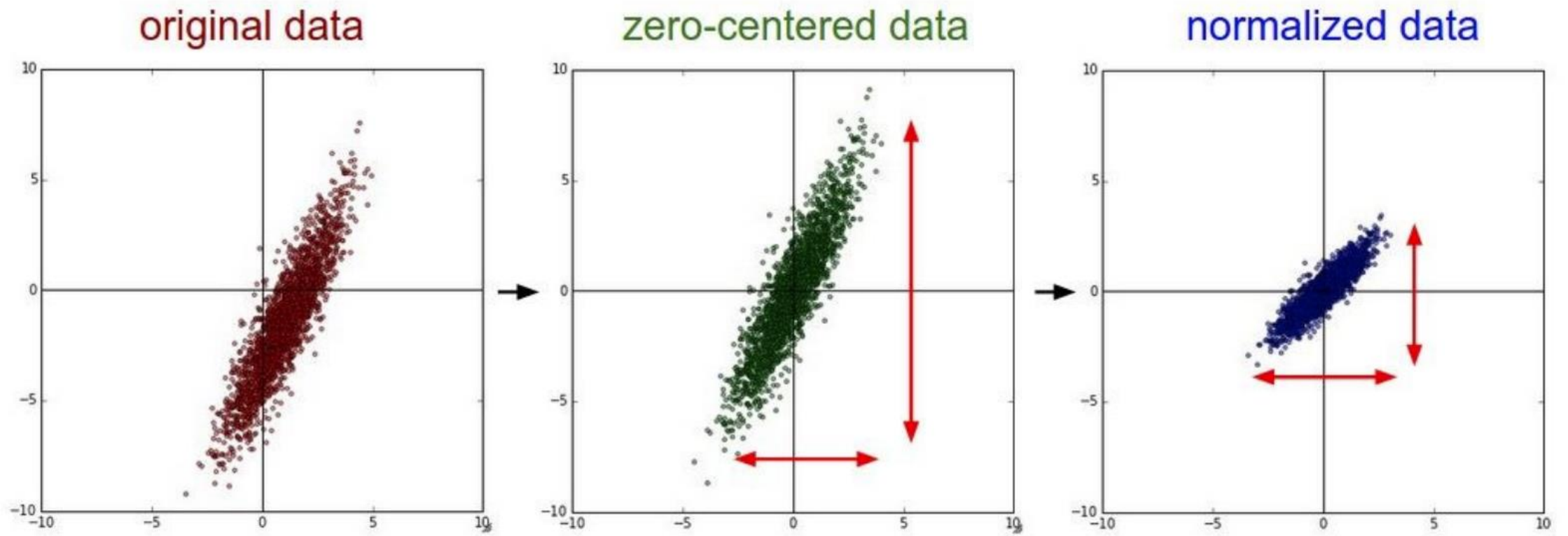
Bias correction

AdaGrad / RMSProp

Bias correction for the fact that
first and second moment
estimates start at zero

Kingma and Ba, "Adam: A method for stochastic optimization", ICLR 2015

https://cs231n.stanford.edu/slides/2024/lecture_3.pdf

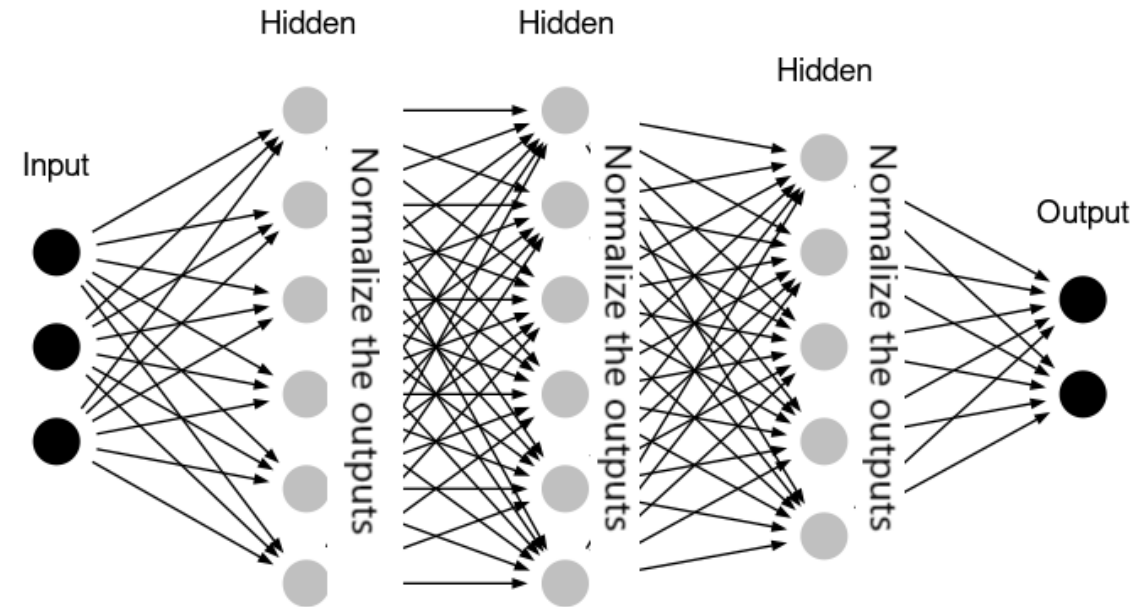


□ **Batch normalization** — It is a step of hyperparameter γ, β that normalizes the batch $\{x_i\}$. By noting μ_B, σ_B^2 the mean and variance of that we want to correct to the batch, it is done as follows:

$$x_i \leftarrow \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

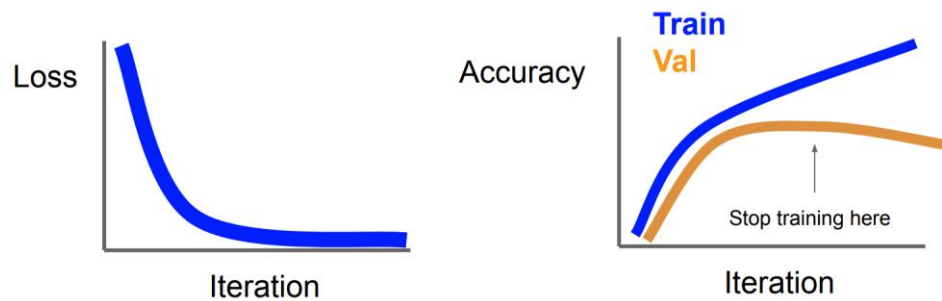
It is usually done after a fully connected/convolutional layer and before a non-linearity layer and aims at allowing higher learning rates and reducing the strong dependence on initialization.

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks>

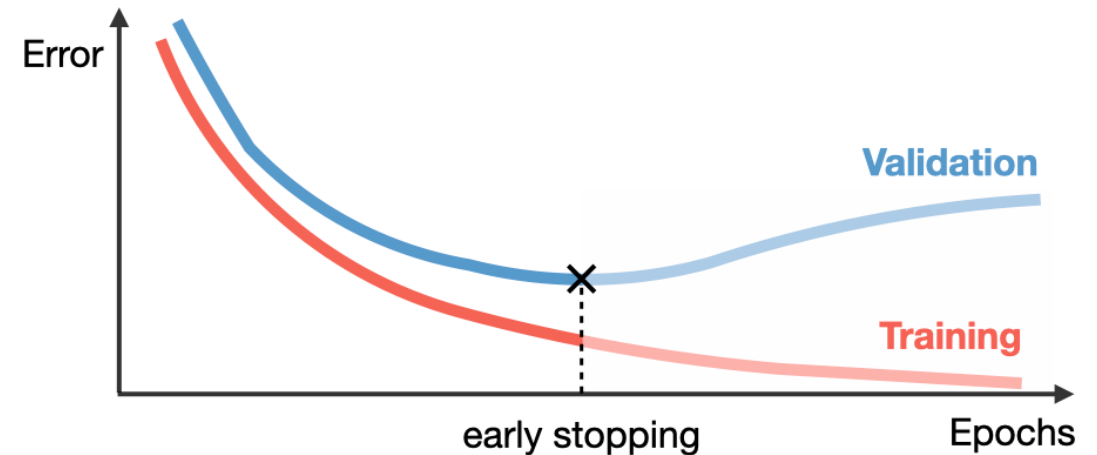


<https://deeptai.org/machine-learning-glossary-and-terms/batch-normalization>

Early Stopping: Always do this



Stop training the model when accuracy on the validation set decreases
Or train for a long time, but always keep track of the model snapshot
that worked best on val

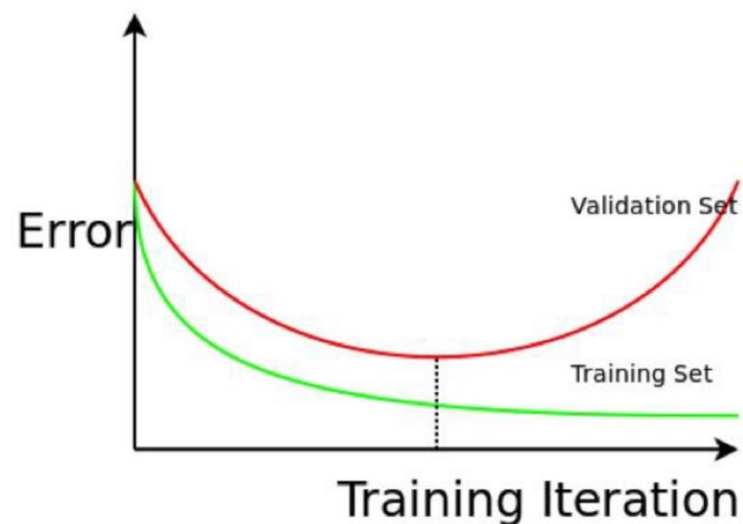


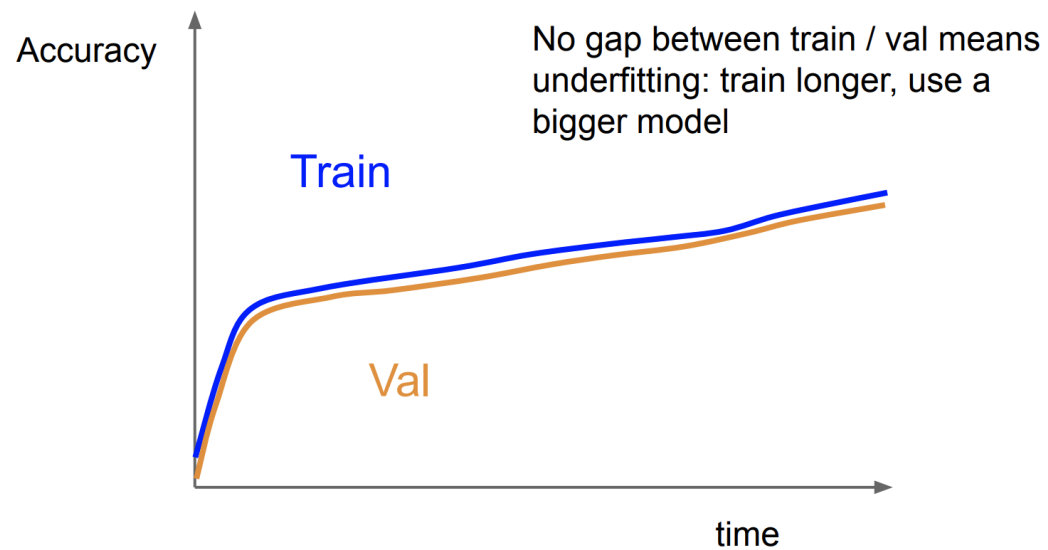
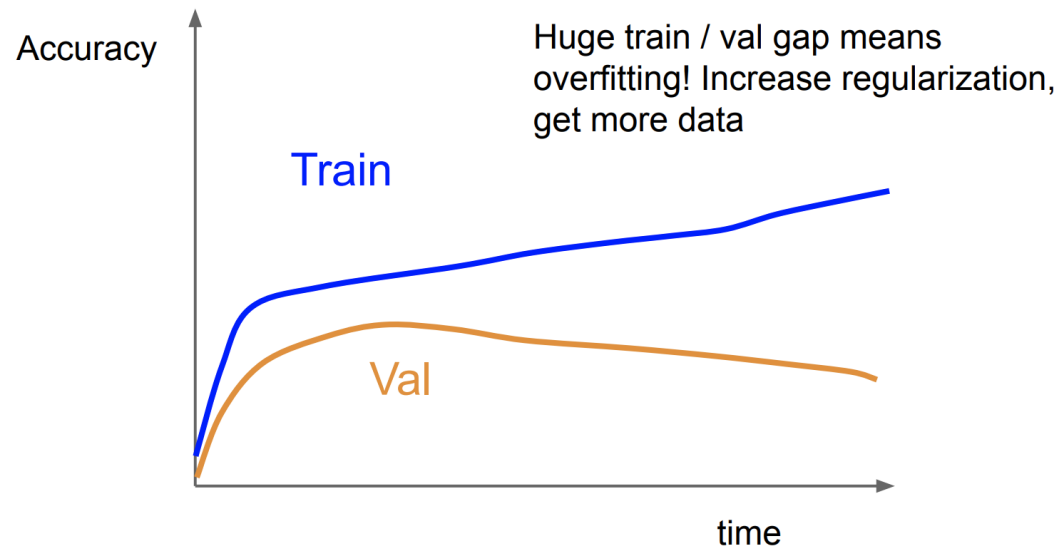
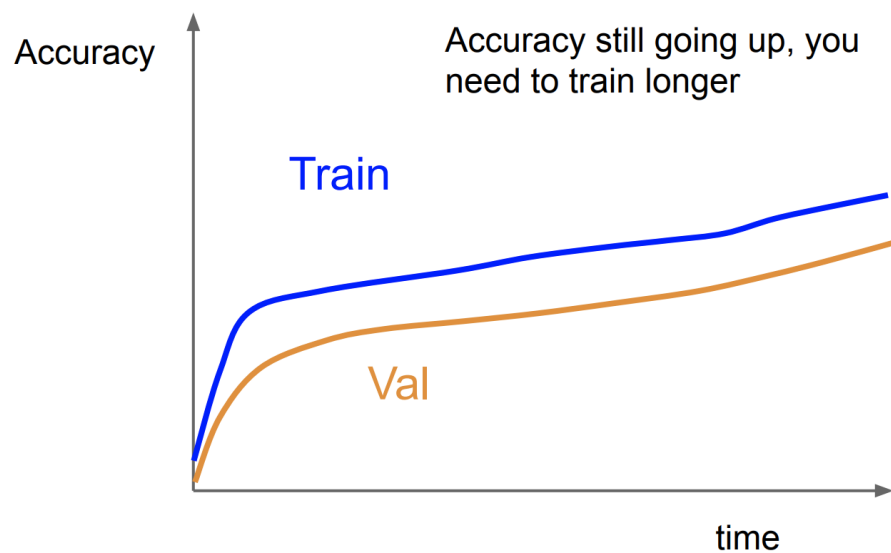
<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks>

http://cs231n.stanford.edu/2021/slides/2021/lecture_8.pdf

Tinklo persimokymas: ankstyvas stabdymas

- Tinklo mokymą **reikia stabdyti** tuomet, kai nors paklaida mokymo duomenims mažėja, bet jau pradeda **didėti paklaida validavimo** (testavimo) duomenims.





Overfitting vs. underfitting: more intuition

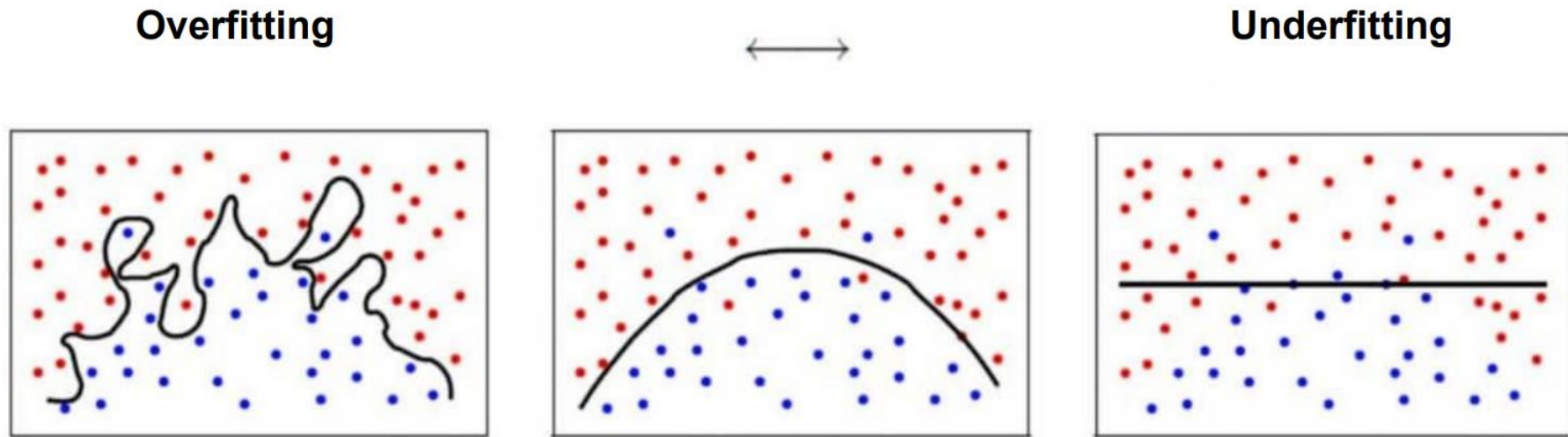


Figure credit: <https://qph.fs.quoracdn.net/main-qimg-412c8556aac7e25b86bba63e9e67ac6-c>

<https://web.stanford.edu/class/archive/biods/biods220/biods220.1204/lectures/lecture2.pdf>